

category: networking-protocol

tags: [tcp-server, heartbeat, x11, xtest, blocking-call, async]

severity: high

created: 2026-07-31

updated: 2026-07-31

project-origin: 6.RemoteControlTool (IPGS.RemoteControl.ZcuAgent — remote control TCP protocol)

Lệnh X11 đồng bộ (XSync) trong vòng đọc message làm "watchdog heartbeat ĐỘC LẬP" tưởng nhầm mất kết nối

Tình huống gặp phải

`ZcuAgent` (server TCP nhận lệnh chuột/bàn phím từ CCU qua `ClientSession.RunReceiveLoopAsync`)

có sẵn kiến trúc heartbeat khá cẩn thận: watchdog timeout (`RunHeartbeatWatchdogAsync`) chạy

task riêng, độc lập với vòng gửi (capture) và vòng nhận (receive) — comment gốc giải

thích rõ lý do: tránh trường hợp watchdog nằm chung capture loop, không bao giờ chạy được

nếu capture loop bị chặn ở `WriteAsync` với 1 reader chậm/độc hại (audit L3).

Tưởng đã an toàn — nhưng vẫn có 1 đường khác khiến watchdog trigger sai: `**receive loop`

tự nó bị chặn**, dù watchdog chạy độc lập, vì watchdog chỉ đọc timestamp `_lastPongTicks`

do CHÍNH receive loop cập nhật — nếu receive loop kẹt, timestamp không bao giờ refresh.

Triệu chứng / Lỗi

Trên ZCU thật (kztek-ZCU2, IP đổi qua 192.168.21.93/96): user báo "remote được nhưng không

điều khiển được" VÀ "cứ bị đã kết nối, mất kết nối" — 2 triệu chứng tưởng không liên

quan, xảy ra cùng lúc, cùng chu kỳ ~15-20s (đúng bằng `PingTimeoutMs`).

Nguyên nhân gốc rễ (Root Cause)

`ClientSession.RunReceiveLoopAsync` xử lý **TẤT CẢ** loại message trong CÙNG 1 vòng lặp

tuần tự, kể cả `Pong` (heartbeat) VÀ `MouseMove/MouseButton/KeyEvent`:

```
case MessageType.MouseMove:
    var (mx, my) = MessageCodec.DecodeMouseMove(payload);
    _injector.Move(mx, my); // ← gọi ĐỒNG BỘ, chặn vòng lặp
    break;
case MessageType.Pong:
    Interlocked.Exchange(ref _lastPongTicks, Environment.TickCount64);
    break;
```

`MouseInjector.Move()` gọi `XTestFakeMotionEvent + XFlush + `XSync(display, false)`` —

`XSync` là round-trip ĐỒNG BỘ chờ X server xác nhận (cố ý thêm để bắt lỗi async ngay, theo

comment gốc). Nếu X server đang bận (ví dụ: `XShmAttach rejected... falling back to

XGetImage — capture màn hình phải dùng cách chậm/tốn tài hơn XShm nhiều), XSync` có thể trễ đủ lâu để CHẶN LUÔN việc đọc message kế tiếp trong loop — kể cả gói Pong sắp tới.

Watchdog (dù chạy task riêng, đúng thiết kế) chỉ thấy `_lastPongTicks` không refresh → tưởng mất kết nối thật → ngắt session sau 15s — dù đường truyền hoàn toàn bình thường.

Giải pháp

Tách lệnh inject (chuột/bàn phím) ra khỏi vòng đọc bằng 1 hàng đợi + task tiêu thụ riêng:

```
private readonly Channel<Action> _inputQueue = Channel.CreateUnbounded<Action>(
    new UnboundedChannelOptions { SingleReader = true, SingleWriter = true });

// Trong RunReceiveLoopAsync — chỉ enqueue, không chặn:
case MessageType.MouseMove:
    _inputQueue.Writer.TryWrite(() => _injector.Move(mx, my));
    break;

// Task riêng — tiêu thụ tuần tự (giữ đúng thứ tự), cảnh báo nếu chậm nhưng KHÔNG chờ:
private async Task RunInputInjectorLoopAsync(Cancellation_token ct)
{
    await foreach (var action in _inputQueue.Reader.ReadAllAsync(ct))
    {
        var task = Task.Run(action, ct);
        if (await Task.WhenAny(task, Task.Delay(InjectWarnMs, ct)) != task)
            _logger.LogWarning("input injection call exceeded {Ms}ms", InjectWarnMs);
    }
}
```

Áp dụng lại (How to reuse)

- Khi 1 vòng đọc/nhận message (network receive loop) VỪA xử lý heartbeat/keepalive VỪA xử lý lệnh gọi API/native blocking (file I/O, X11/Win32 call, DB query đồng bộ...) — PHẢI tách các lệnh có khả năng chậm/blocking ra khỏi vòng đọc, kể cả khi heartbeat check đã chạy ở task riêng — task riêng không cứu được nếu nó phụ thuộc vào 1 giá trị mà CHỈ vòng đọc đang bị kẹt mới cập nhật.
- 2 triệu chứng "input không có tác dụng" + "kết nối chập chờn" xuất hiện CÙNG LÚC, CÙNG CHU KỲ với timeout config → nghi ngay đến việc xử lý input đang chặn đường đọc heartbeat, đừng coi là 2 bug độc lập.
- XSync/bất kỳ API đồng bộ chờ phản hồi thiết bị ngoài (X server, driver, hardware) đều có khả năng trễ bất định dưới tải cao — không nên gọi trực tiếp trong hot path xử lý network protocol.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Dùng `Channel<Action>` với closure bắt biến local (`mx, my, sx, sy...`) — phải đảm bảo các biến đó không bị mutate sau khi tạo closure (ở đây an toàn vì đều là giá trị cục bộ decode ra ngay trước khi enqueue, không bị ghi đè).

- ⚠ Không `await` task trong `RunInputInjectorLoopAsync` khi nó chạy quá `InjectWarnMs` — chỉ log cảnh báo rồi tiếp tục đọc queue tiếp; native call chậm vẫn chạy ngầm tới khi xong, không bị "giết" — chấp nhận được vì XTest thường không treo vĩnh viễn, chỉ trễ tạm thời.

Tham chiếu

- Project liên quan: `6.RemoteControlTool` — `IPGS.RemoteControl.ZcuAgent/Net/ClientSession.cs` (F25)
- Liên quan: comment "audit L3" đã có sẵn trong code — bài học tương tự (blocking write chặn heartbeat) nhưng ở chiều send, không phải receive